百科荣创

BECSTRONG

EXPERIMENT GUIDE BOOK | 实验指导书

百科荣创（北京）科技发展有限公司

机器学习应用课程资源

前 言

百科荣创成立于 2003 年，是国内领先的新一代信息技术产业高素质工程技术与技术技能人才培养解决方案提供商，设有北京总部、济南分部（产教融合服务中心、企业公共实习实践基地）、深圳分部（产品研发中心）、成都分部（筹），国家级高新技术企业与双软认证企业，国内电子、通信、嵌入式、物联网、人工智能等电子信息领域教育头部企业，面向全国高校、职业院校提供专业共建、人才联合培养、实验实训装备、课程开发、教材联合开发、学生实习实践与就业等产品与产教融合服务。公司自成立以来，坚持秉承“产品就是人品！做企业就是做人！”的经营理念和“来源于教育，回报于教育”的企业精神，汇聚了一大批电子、通信、嵌入式、物联网、人工智能方向的优秀人才，以培养智能化时代电子信息大类复合型技术技能人才为宗旨，致力于为客户提供最满意的产品和服务，力争成为中国高等教育与职业教育领域的产教融合型领军企业。

十多年来，公司产品行销全国各省、市、自治区，成为众多“双一流”高校以及职业院校采购同类产品的首选，用户已遍及清华大学、北京大学、北京理工大学、北京科技大学、天津大学、重庆大学、东北大学、四川大学、深圳职业技术学院、天津电子信息职业技术学院、上海电子信息职业技术学院、北京电子科技职业学院等全国各大知名院校。

为了给您提供更加优质的服务，公司设有 400 开头的免费电话，在使用过程中，如果您有任何问题需要咨询或者需要服务，请与本公司相关部门联系，我们定当竭诚为您服务。服务电话：400-067-7899。

欢迎您在使用的过程中为我们提出宝贵的意见和建议。

目 录

1.1 回归算法----------1
1.2 聚类算法----------6
1.3 分类算法----------15
1.4 决策树----------22
1.5 支持向量机 SVM ----------29

1.1 回归算法

1.1.1 实验目的

1. 学习回归的基本概念。
2. 熟悉回归的常用算法。
3. 了解回归算法的评价标准。
4. 学会使用 scikit-learn API 函数实现回归算法。

1.1.2 实验内容

使用 scikit-learn API 中的回归算法实现房屋价格回归分析。

1.1.3 实验原理

1. 回归简介

回归：统计学分析数据的方法，目的在于了解多个数据变量间是否相关、研究其相关方向与强度，并建立数学模型以便观察特定变数来预测研究者感兴趣的变数。回归分析可以帮助人们了解在自变量变化时应变量的变化。通过回归分析我们可以由自变量估计因变量的条件期望。

2. 回归技术

线性回归的目的是预测数值型的目标值。最直接的方式是依据输入写出一个目标值计算公式，其表达形式为 y = w’x+e，e 为误差服从均值为 0 的正态分布。回归分析中，只包括一个自变量和一个因变量，且二者的关系可用一条直线近似表示，这种回归分析称为一元线性回归分析。如果回归分析中包括两个或两个以上的自变量，且因变量和自变量之间是线性关系，则称为多元线性回归分析。

线性回归得出的模型不一定是一条直线：

1) 在只有一个变量的时候，模型是平面中的一条直线；
2) 有两个变量的时候，模型是空间中的一个平面；
3) 有更多变量时，模型将是更高维的。

3. 常用回归算法

在机器学习中常用的回归算法主要有三种：线性回归、逻辑回归、正则化。

1) 线性回归

假定输入变量（X）和单个输出变量（Y）之间呈线性关系。它旨在找到预测值 Y 的线性方程：Y=WX+b 其中，X=(x1,x2,...,xn)为 n 个输入变量，W=(w1,w2,...,wn)为线性系数，b 是偏置项。目标是找到系数 W 的最佳估计，使得预测值 Y 的误差最小。使用最小二乘法估计线性系数 W，即使预测值(Yhat) 与观测值(Y)之间的差的平方和最小。其中线性回归又可分简单线性回归：Y=wX+b、多元线性回归：Y=w1x1+w2x2+…+wnxn+b、多项式回归：Y=b0+b1x1+b2x1^2+b3x1^3+…+bnx1^n，这个表达式和多元线性回归非常像，唯一的区别就是多项式线性回归中存在很多次方项，而多元线性回归中是多个变量。

2) 逻辑回归

用来确定一个事件的概率。通常来说，事件可被表示为类别因变量。事件的概率用 logit 函数（Sigmoid 函数）表示，逻辑回归用于分类问题，例如，对于给定的医疗数据，可以使用逻辑回归判断一个人是否患有癌症。如果输出类别变量具有两个或更多个层级，则可以使用多项式逻辑回归。

3) 正则化

当有大量的输入特征时，需要正则化来确保预测模型不会太复杂。正则化可以帮助防止数据过拟合。它也可以用来获得一个凸损失函数。有两种类型的正则化——L1 和 L2 正则化。当数据高度共线时，L1 正则化也可以工作。在 L1 正则化中，与所有系数的绝对值的和相关的附加惩罚项被添加到损失函数中。L2 正则化提供了稀疏的解决方案。当输入特征的数量非常大时，非常有用。

4. 回归评价标准

回归算法的评价指标常用的有 MSE，RMSE，MAE、R-Squared。

MSE（Mean Squared Error）均方误差，用真实值减去预测值 然后平方之后求和平均。线性回归用 MSE 作为损失函数。

RMSE（Root Mean Squared Error）均方根误差，这就是 MSE 开个根号。其实实质是一样的，可用于数据更好的描述。

MAE（Mean absolute Error）平均绝对误差

R-Squared 方差除了可以预测数据准确性外，还可以捕捉到数据的”规律“，比如数据的分布规律，单调性等。

RMSLE(Root Mean Squared Logarithmic Error) 当数据当中有少量的值和真实值差值较大的时候，使用该函数能够减少这些值对于整体误差的影响。

MSE 和 MAE 适用于误差相对明显的时候，大的误差也有比较高的权重，RMSE 则是针对误差不是很明显的时候;MAE 是一个线性的指标,所有个体差异在平均值上均等加权，所以它更加凸显出异常值，相比 MSE；RMSLE： 主要针对数据集中有一个特别大的异常值，这种情况下，data 会被 skew，RMSE 会被明显拉大，这时候就需要先对数据 log 下，再求 RMSE，这个过程就是 RMSLE。对低估值（under-predicted）的判罚明显多于估值过高(over-predicted)的情况（RMSE 则相反）。

5. 房屋价格回归分析

我们知道与房价密切相关的除了单位的房价，还有房屋的尺寸。我们可以根据已知的房屋成交价和房屋的尺寸进行线性回归，继而可以对已知房屋尺寸，而未知房屋成交价的实例进行成交价格的预测。具体做法是对房屋成交信息建立回归方程，并依据回归方程对房屋价格进行预测。

首先我们来看看房屋实例数据，进行可行性分析。为了方便展示，成交信息只使用了房屋的面积以及对应的成交价格。其中房屋面积单位为平方英尺，房屋成交价格单位为万。部分数据如图 1.1.1 所示：

编号	房屋面积/ft^2	交易价格/万	编号	房屋面积/ft^2	交易价格/万
1	1000	168	26	2700	285
2	792	184	27	2612	292
3	1260	197	28	2705	482
4	1262	220	29	2570	462
5	1240	228	30	2442	352
6	1170	248	31	2387	440
7	1230	305	32	2292	462
8	1255	256	33	2308	325
9	1194	240	34	2252	298
10	1450	230	35	2202	352
11	1481	202	36	2157	403
12	1475	220	37	2140	308
13	1482	232	38	4000	795
14	1484	460	39	4200	765
15	1512	320	40	3900	705
16	1680	340	41	3544	420
17	1620	240	42	2980	402
18	1720	368	43	4355	762
19	1800	280	44	3150	392
20	4400	710	45	3025	320
21	4212	552	46	3450	350
22	3920	580	47	4402	820
23	3212	585	48	3454	425
24	3151	590	49	890	272

图 1.1.1 房屋实例数据

对实例数据进行可行性分析，通过数据可视化直接观察房屋成交价格与房屋尺寸间是否存在线性关系。这里使用散点图将其在二维平面上进行可视化表现。

图 1.1.2 为数据散点图，其中横坐标为房屋面积，纵坐标为房屋的成交价格。可以看出，靠近坐标坐下角部分的点，表示房屋尺寸较小的房子，其对应的房屋成交价格也相对较低。同样的，靠近坐标右上角部分的点对应于大尺寸高价格的房子，从总体来看，

房屋面积和成交价格基本成正比。

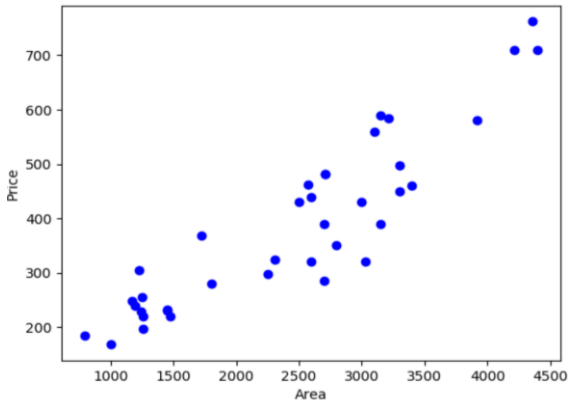


图 1.1.2 数据散点图

1.1.4 实验步骤

通过 sklearn 库中的线性回归来拟合房屋尺寸和房价的线性关系。

1） 建立工程并导入 sklearn 包，新建一个 house.py 文件，导入 sklearn 相关包。

```
import matplotlib.pyplot as plt
from sklearn import linear_model    #导入 sklearn 中 linear_model 模块进行线性回归
```

2） 加载训练数据，建立回归方程。

```
datasets_X = []    #  用来储存房屋尺寸数据
datasets_Y = []    #  用来储存房价
fr = open('prices.txt', 'r')            #  以只读的方式读取数据集所在的文件 prices.txt
lines = fr.readlines()                  #  一次性读取所有的数据集，返回一个列表
for line in lines:                      #  逐行遍历
    items = line.strip().split(',')    #  去除数据集中的不可见字符，并用逗号分割数据
    datasets_X.append(int(items[0]))   #  将读取的数据转换为 int 型，并分别写入
datasets_X 和 datasets_Y
    datasets_Y.append(int(items[1]))

length = len(datasets_X)    #  求数据集的总长度

datasets_X = np.array(datasets_X).reshape([length, 1])    #  将 datasets_X 转化为数
组，并变为二维，以符合线性回归拟合函数输入参数要求
datasets_Y = np.array(datasets_Y)

"""
    以数据 datasets_X 的最大值和最小值为范围，
    建立等差数列，方便后续画图
```

```
"""
minX = min(datasets_X)
maxX = max(datasets_X)
X = np.arange(minX,maxX).reshape([-1,1])

"""
    degree=2 表示建立 datasets_X 的二次多项式特征 X_poly。
    然后创建线性回归，使用线性模型学习 X_poly 和 datasets_Y 之间的映射关系（即参数）。
"""
poly_reg = PolynomialFeatures(degree = 2)
X_poly = poly_reg.fit_transform(datasets_X)
lin_reg_2 = linear_model.LinearRegression()
lin_reg_2.fit(X_poly, datasets_Y)
```

3） 回归拟合结果可视化。

```
plt.scatter(datasets_X, datasets_Y, color = 'blue')   # 标签数据散点图
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)),color = 'red')   # 绘制回归拟合的出来的线
plt.xlabel('Area')
plt.ylabel('Price')
plt.show()
```

4） 通过多项式回归拟合的曲线与数据点的关系如图 1.1.3 所示。依据该多项式回归方程即可通过房屋的尺寸，来预测房屋的成交价格。

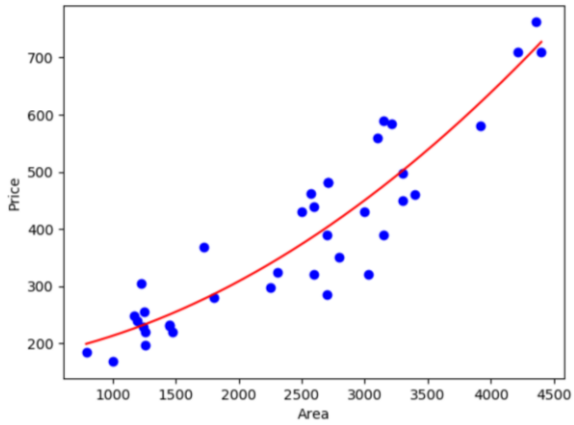


图 1.1.3 回归结果展示

1.1.5 思考题

1. 简述常用的回归算法有哪些？
2. 使用现有的回归算法解决非线性回归问题。

1.2 聚类算法

1.2.1 实验目的

1. 学习聚类的基本概念。
2. 熟悉回归的常用算法。
3. 了解聚类算法的评价标准。
4. 学会使用 scikit-learn API 函数实现聚类算法。

1.2.2 实验内容

使用 scikit-learn API 中的聚类算法实现家庭消费水平聚类分析。

1.2.3 实验原理

1. 聚类简介

聚类(clustering)，就是根据数据的“相似性”将数据分为多类的过程。评估两个不同样本之间的“相似性”，通常使用的方法就是计算两个样本之间的“距离”。使用不同的方法计算样本间的距离会关系到聚类结果的好坏。聚类效果如图 1.2.1 所示：

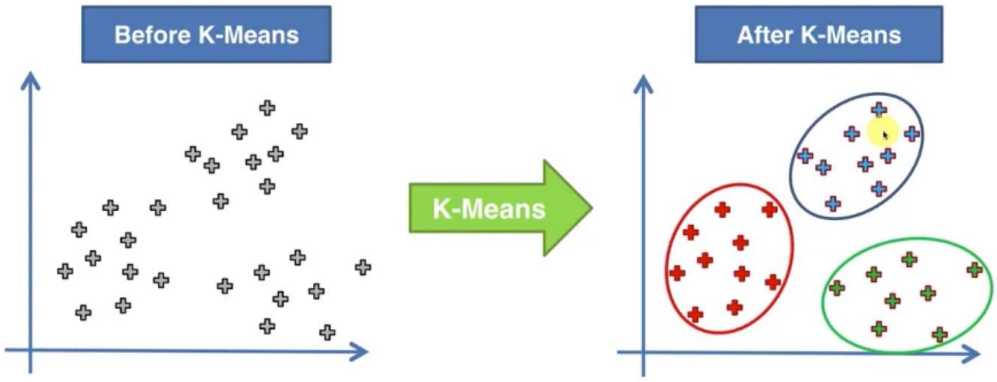


图 1.2.1 聚类图示

2. 聚类技术

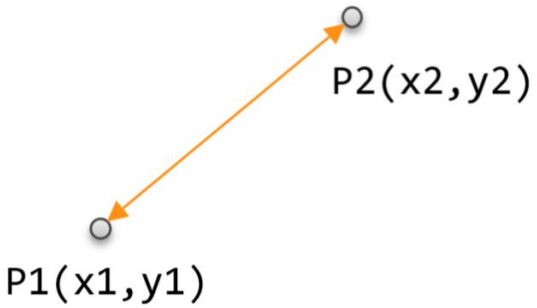
物以类聚，人以群分，是对聚类最好的解释。聚类直观上来说是将相似的样本聚在一起，从而形成一个类簇（cluster）。评估两个不同样本之间的“相似性”，通常使用的方法就是计算两个样本之间的“距离”。计算样本之间地距离通常有以下几种。

欧氏距离是最常用的一种距离度量方法，源于欧式空间中两点的距离。其计算方法如图 1.2.2 所示：

$$d = \sqrt{\sum_{k=1}^{n}(x_{1k} - x_{2k})^2}$$

图 1.2.2 欧式距离计算公式

二维空间中的欧氏距离的计算示意如图 1.2.3 所示：



$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

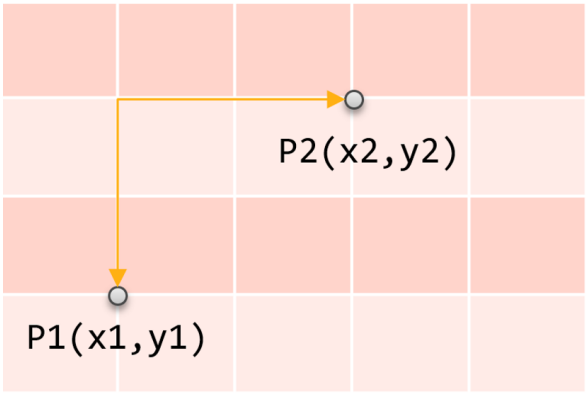
图 1.2.3 二维空间欧氏距离计算示意

曼哈顿距离也称作“城市街区距离”，类似于在城市之中驾车行驶，从一个十字路口到另外一个十字楼口的距离。其计算方法如图 1.2.4 所示：

$$d = \sum_{k=1}^{n}|x_{1k} - x_{2k}|$$

图 1.2.4 曼哈顿距离计算公式

二维空间中曼哈顿距离计算示意如图 1.2.5 所示：



$$d = |x_1 - x_2| + |y_1 - y_2|$$

图 1.2.5 曼哈顿距离计算示意图

马氏距离表示数据的协方差距离，是一种尺度无关的度量方式。也就是说马氏距离会先将样本点的各个属性标准化，再计算样本间的距离。其计算方式如图 1.2.6 所示：

$$d(x_i, x_j) = \sqrt{(x_i - x_j)^T s^{-1} (x_i - x_j)}$$

图 1.2.6 马氏距离计算公式

二维空间的马氏距离如图 1.2.7 所示：

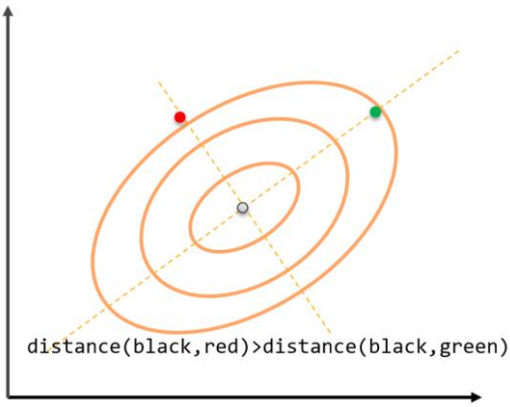


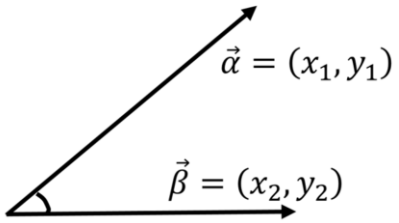
图 1.2.7 二维空间中的马氏距离

余弦相似度用向量空间中两个向量夹角的余弦值作为衡量两个样本差异的大小。余弦值越接近 1，说明两个向量夹角越接近 0 度，表明两个向量越相似。其计算方法如图 1.2.8 所示：

$$\boldsymbol{cos(\theta)} = \frac{\sum_{k=1}^{n} \boldsymbol{x}_{1k}\boldsymbol{x}_{2k}}{\sqrt{\sum_{k=1}^{n} \boldsymbol{x}_{1k}^2}\sqrt{\sum_{k=1}^{n} \boldsymbol{x}_{2k}^2}}$$

图 1.2.8 余弦夹角计算工程

二维空间中夹角余弦如图 1.2.9 所示：



$$\boldsymbol{cos(\theta)} = \frac{\boldsymbol{x}_1\boldsymbol{x}_2 + \boldsymbol{y}_1\boldsymbol{y}_2}{\sqrt{\boldsymbol{x}_1^2 + \boldsymbol{y}_1^2} \cdot \sqrt{\boldsymbol{x}_2^2 + \boldsymbol{y}_2^2}}$$

图 1.2.9 二维空间中的夹角余弦示意图

3. 典型聚类算法

K-means、DBSCAN 是较为经典的聚类算法。k-means 算法以 k 为参数，把 n 个对象分成 k 个簇，使簇内具有较高的相似度，而簇间的相似度较低。而 DBSCAN 算法是一种基于密度的聚类算法，聚类的时候不需要预先指定簇的个数，最终的簇的个数不定。

K-means 的一般过程如图 1.2.10 所示：

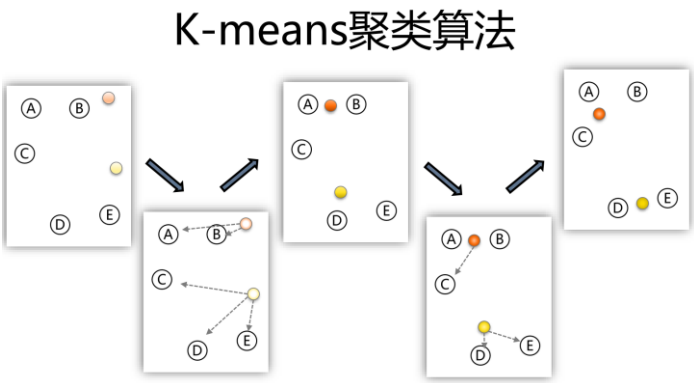


图 1.2.10 K-means 一般处理过程

1) 随机选择 k 个点作为初始的聚类中心；
2) 对于剩下的点，根据其与聚类中心的距离，将其归入最近的簇；
3) 对每个簇，计算所有点的均值作为新的聚类中；

4） 重复 2、3 直到聚类中心不再发生改变。

DBSCAN 算法将数据点分为三类，如图 1.2.11 所示：

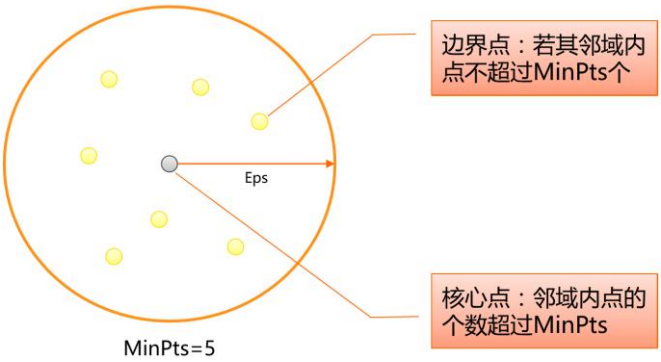


图 1.2.11 DBSCAN 算法

核心点：在半径 Eps 内含有超过 MinPts 数目的点；边界点：在半径 Eps 内点的数量小于 MinPts，但是落在核心点的邻域内；噪音点：既不是核心点也不是边界点的点。

DBSCAN 算法流程：

1） 将所有点标记为核心点、边界点或噪声点；

2） 删除噪声点；

3） 为距离在 Eps 之内的所有核心点之间赋予一条边；

4） 每组连通的核心点形成一个簇；

5） 将每个边界点指派到一个与之关联的核心点的簇中(哪一个核心点的半径范围之内)。

使用 DBSCAN 算法对如下 13 个样本点进行聚类，取 Eps=3，MinPts=3，依据 DBSACN 对所有点进行聚类（曼哈顿距离）。

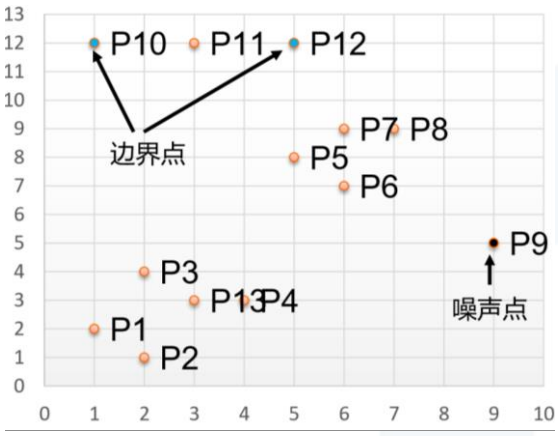


图 1.2.12 划分三类点

对每个点计算其邻域 Eps=3 内 的点的集合。集合内点的个数超过 MinPts=3 的点为核心点，查看剩余点是否在核心点的邻域内，若在，则为边界点，否则为噪声点。

将距离不超过 Eps=3 的点相互连接，构成一个簇，核心点邻域内的点也会被加入到这个簇中。则形成 3 个簇。

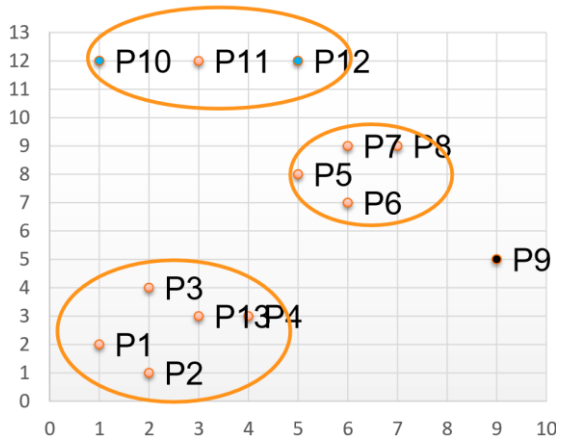


图 1.2.13 DBSCAN 算法聚类效果

4. 聚类算法评价标准

对聚类效果进行评价的研究称为聚类有效性分析，在聚类有效性分析研究中，评价聚类算法得到的聚类效果的方法主要有三类。

1) 外部标准：用事先判定的聚类结构来评价，与某专家给出的参考模型进行对比。
2) 内部标准：用参与聚类的样本（n 个数据对象）来评价，如采用各簇误差平方和。
3) 相对标准：用同一算法不同结果（不同参数得到）来评价，通过与其他结果的比较来判断评价的优劣性。

聚类分析中，存在两种相互联系的评价标准。聚类结果中，簇类越紧密，簇间分离就越好。聚类结果与人工判定结果越吻合越好。

5. 家庭消费水平聚类分析

现有 1999 年全国 31 个省份城镇居民家庭平均每人全年消费性支出的八个主要变量数据，这八个变量分别是：食品、衣着、家庭设备用品及服务、医疗保健、交通和通讯、娱乐教育文化服务、居住以及杂项商品和服务。如图 1.2.14 所示：

城市	食品	衣着	家庭设备用品及服务	医疗保健	交通和通讯	娱乐教育文化服务	居住	杂项商品和服务
北京	2959	730.79	749.41	513.34	467.87	1141.82	478.42	457.64
天津	2460	495.47	697.33	302.87	284.19	735.97	570.84	305.08
河北	1496	515.9	362.37	285.32	272.95	540.58	364.91	188.63
山西	1406	477.77	290.15	208.57	201.5	414.72	281.84	212.1
内蒙古	1304	524.29	254.83	192.17	249.81	463.09	287.87	192.96
辽宁	1731	553.9	246.91	279.81	239.18	445.2	330.24	163.86
吉林	1562	492.42	200.49	218.36	220.69	459.62	360.48	147.76
黑龙江	1410	510.71	211.88	277.11	224.65	376.82	317.61	152.85
上海	3712	550.74	893.37	346.93	527	1034.98	720.33	462.03
江苏	2208	449.37	572.4	211.92	302.09	585.23	429.77	252.54
浙江	2629	557.32	689.73	435.69	514.66	795.87	575.76	323.36
安徽	1845	430.29	271.28	126.33	250.56	513.18	314	151.39
福建	2709	428.11	334.12	160.77	405.14	461.67	535.13	232.29
江西	1564	303.65	233.81	107.9	209.7	393.99	509.39	160.12
山东	1676	613.32	550.71	219.79	272.59	599.43	371.62	211.84
河南	1428	431.79	288.55	208.14	217	337.76	421.31	165.32
湖南	1942	512.27	401.39	206.06	321.29	697.22	492.6	226.45
湖北	1783	511.88	282.84	201.01	237.6	617.74	523.52	182.52
广东	3055	353.23	564.56	356.27	811.88	873.06	1082.82	420.81
广西	2034	300.82	338.65	157.78	329.06	621.74	587.02	218.27
海南	2058	186.44	202.72	171.79	329.65	477.17	312.93	279.19
重庆	2303	589.99	516.21	236.55	403.92	730.05	438.41	225.8
四川	1974	507.76	344.79	203.21	240.24	575.1	430.36	223.46
贵州	1674	437.75	461.61	153.32	254.66	445.59	346.11	191.48
云南	2194	537.01	369.07	249.54	290.84	561.91	407.7	330.95
西藏	2647	839.7	204.44	209.11	379.3	371.04	269.59	389.33
陕西	1473	390.89	447.95	259.51	230.61	490.9	469.1	191.34
甘肃	1526	472.98	328.9	219.86	206.65	449.69	249.66	228.19
青海	1655	437.77	258.78	303	244.93	479.53	288.56	236.51
宁夏	1375	480.89	273.84	317.32	251.08	424.75	228.73	195.93
新疆	1609	536.05	432.46	235.82	250.28	541.3	344.85	214.4

图 1.2.14 1999 年全国 31 个省份城镇居民家庭平均每人全年消费性支出数据

1.2.4 实验步骤

通过 K-means 聚类算法对城镇居民数据进行分析，可以了解 1999 年各个省份的消费水平在国内的情况。

1） 建立工程，导入 sklearn 相关包。

```
import numpy as np
# 使用 sklearn.cluster.KMeans 可以调用 K-means 算法进行聚类
from sklearn.cluster import Kmeans
```

2） 加载数据，创建 K-means 算法实例，并进行训练，获得标签。

```
"""
    从指定文件路径中加载城镇居民家庭全年消费水平
    返回值：返回城市名称，以及该城市的各项消费信息
"""
def loadData(filePath):
    fr = open(filePath,'r+')
    lines = fr.readlines()
    retData = []
    retCityName = []
    for line in lines:
        items = line.strip().split(",")
        retCityName.append(items[0])
        retData.append([float(items[i]) for i in range(1,len(items))])
    #  retCityName：用来存储城市名称  retData：用来存储城市的各项消费信息
```

```
        return retData,retCityName

    if __name__ == '__main__':
        data,cityName = loadData('city.txt')   # 加载数据集
        km = KMeans(n_clusters=4)   # 创建 K-means 实例，n_clusters 用于指定聚类中心的个数
        label = km.fit_predict(data)   #计算簇中心以及为簇分配序号
        expenses = np.sum(km.cluster_centers_,axis=1) # 聚类中心点的数值加和，也就是平均消费水平
```

3） 输出标签，结果输出。

```
        print(expenses)
        CityCluster = [[],[],[],[]]   # 四档消费水平
        for i in range(len(cityName)):
            CityCluster[label[i]].append(cityName[i])   # 将城市按 label 分成设定的簇
        for i in range(len(CityCluster)):   # 将每个簇的城市输出 将每个簇的平均花费输出
            print("Expenses:%.2f" % expenses[i])
            print(CityCluster[i])
```

将城市按照消费水平 n_clusters 类，消费水平相近的城市聚集在一类中。聚成 2 个簇：

```
    Expenses:4040.42
    ['河北', '山西', '内蒙古', '辽宁', '吉林', '黑龙江', '江苏', '安徽', '江西', '山东', '河南', '湖南', '湖北', '广西', '海南', '四川', '贵州', '云南', '陕西', '甘肃', '青海', '宁夏', '新疆']
    Expenses:6457.13
    ['北京', '天津', '上海', '浙江', '福建', '广东', '重庆', '西藏']
```

聚成 3 个簇：

```
    Expenses:5113.54
    ['天津', '江苏', '浙江', '福建', '湖南', '广西', '海南', '重庆', '四川', '云南', '西藏']
    Expenses:3827.87
    ['河北', '山西', '内蒙古', '辽宁', '吉林', '黑龙江', '安徽', '江西', '山东', '河南', '湖北', '贵州', '陕西', '甘肃', '青海', '宁夏', '新疆']
    Expenses:7754.66
    ['北京', '上海', '广东']
```

聚成 4 个簇：

```
    Expenses:4357.67
    ['安徽', '湖南', '湖北', '广西', '海南', '四川']
    Expenses:3788.76
```

```
['河北', '山西', '内蒙古', '辽宁', '吉林', '黑龙江', '江西', '山东', '河南', '贵州', '陕西', '甘肃', '青海', '宁夏', '新疆']
Expenses:7754.66
['北京', '上海', '广东']
Expenses:5477.90
['天津', '江苏', '浙江', '福建', '重庆', '云南', '西藏']
```

从结果可以看出消费水平相近的省市聚集在了一类，例如消费最高的北京、上海、广东聚集在了消费最高的类别。聚 4 类时，结果可以比较明显的看出消费层级。

1.2.5 思考题

1. 简述常用的聚类算法有哪些？
2. 将上述案例的输出结果可视化出来。

1.3 分类算法

1.3.1 实验目的

1. 学习分类算法的基本概念。
2. 熟悉常用的分类算法。
3. 了解分类算法的评价标准。
4. 学会使用 scikit-learn API 函数实现分类算法。

1.3.2 实验内容

使用 scikit-learn API 中的分类算法实现新闻分类应用。

1.3.3 实验原理

1. 分类简介

监督学习可以分为两类：分类和回归。分类问题预测数据所属的类别；分类的例子包括垃圾邮件检测、客户流失预测、情感分析、犬种检测等。回归问题根据先前观察到的数据预测数值；回归的例子包括房价预测、股价预测、身高-体重预测等。

分类问题属于预测任务，就是通过已有数据集（训练集）的学习，得到一个目标函数 f（模型），把每个属性集 x 映射到目标属性 y（类），且 y 必须是离散的（若 y 为连续的，则属于回归算法）。

2. 常用分类算法

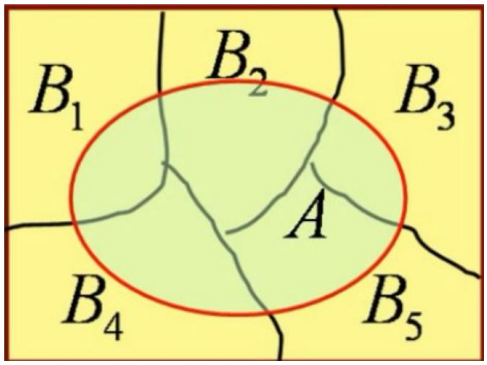
分类在机器学习中的应用非常重要，因为计算机其实无法深层次地理解文字图片目标的意思，只能回答是或者不是。所以让计算机知道 数据的含义是很重要的。机器学习中常用的分类算法有：朴素贝叶斯、决策树、逻辑回归、最近邻 KNN、支持向量机 SVM。

1） 朴素贝叶斯

要理解朴素贝叶斯分类器，需要理解贝叶斯定理，首先要解释的就是条件概率，P（A|B）表示事件 B 发生 的情况下，事件 A 发生的概率。

$$P(A\,|\,\mathrm{B}) = \frac{P(AB)}{P(B)}$$

在解释一下全概率公式，全概率公式为概率论中的重要公式，它将对一复杂事件 A 的概率求解问题转化为了在不同情况下发生的简单事件的概率的求和问题。



如果事件 B1、B2、B3…Bn 构成一个完备事件组，即它们两两互不相容，其和为全集；并且 P（Bi)大于 0，则对任一事件 A 有:

$$P(A)=\sum_{i=1}^{n}P(B_i)P(A\mid B_i)$$

根据条件概率公式和全概率公式可以推导出贝叶斯公式：

$$P(\mathrm{B}\mid\mathrm{A})=\frac{P(A\mid B)\,\mathrm{P}(\mathrm{B})}{P(\mathrm{A})}$$

上面通俗地解释一下原理，对于给出的待分类项，求解在此项出现的条件下各个类别出现的概率，哪个最大，就认为此待分类项属于哪个类别。朴素贝叶斯的定义如下：

(1) 设 x={a1,a2,a3,a4,a5,….,am}为一个代分类项，而每个 a 为 x 的一个属性。

(2) 类别集合 C={y1,y2,y3,y4,…,yn}。

(3) 计算

$$P(\mathrm{y}_1\mid\mathrm{x}),\mathrm{P}(\mathrm{y}_2\mid\mathrm{x}),\ldots,\mathrm{P}(\mathrm{y}_n\mid\mathrm{x})$$

(4) 如果

$$P(\mathrm{y}_k)=\max\{P(\mathrm{y}_1\mid\mathrm{x}),\mathrm{P}(\mathrm{y}_2\mid\mathrm{x}),\ldots,\mathrm{P}(\mathrm{y}_n\mid\mathrm{x})\}$$

(5) 那么 x 就属于y_k。

第三步计算各个条件概率，其实就是朴素贝叶斯分类器训练的过程。

总结一些朴素贝叶斯的优缺点，优点：对小规模的数据表现很好，适合多分类任务，适合增量式训练。缺点：对输入数据的表达形式很敏感。

2） 决策树

决策树（decision tree）是一种树形结构的分类器，通过顺序询问分类点的属性决定分类点最终的类别。通常根据特征的信息增益或其他指标，构建一颗决策树。在分类时，只需要按照决策树中的结点依次进行判断，即可得到样本所属类别。例如，根据右图这个构造好的分类决策树，一个无房产，单身，年收入 55K 的人的会被归入无法偿还信用卡这个类别。

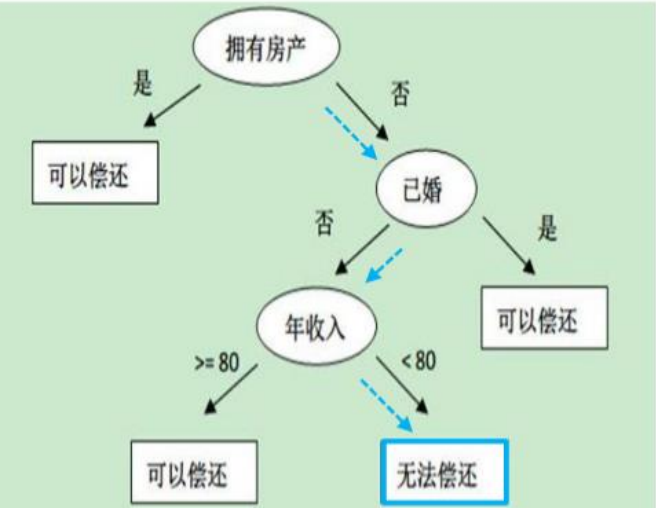


图 1.3.1 信用卡偿还能力决策树分类

3） 逻辑回归（Logistic Regression(LR)）

Logistic Regression 别看它名字里带了回归，但是它其实是一种用于分类的方法，用于解决两分类问题。逻辑回归仅在线性回归的基础上，套用了一个逻辑函数（sigmoid），sigmoid 函数公式为：

$$g(x) = \frac{1}{1 + e^{-x}}$$

函数曲线图像为图 1.3.2 所示：

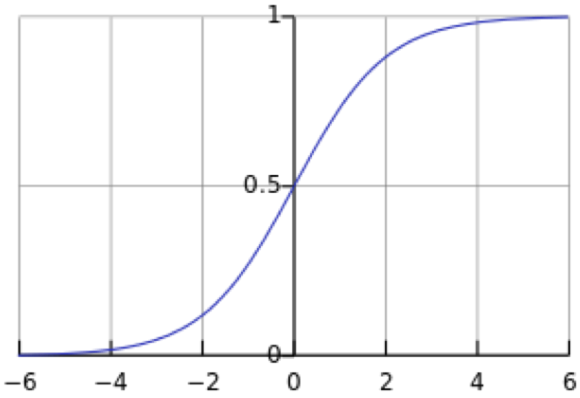


图 1.3.2 sigmoid 函数曲线

由于这个逻辑函数，逻辑回归成为了机器学习领域一颗耀眼的明星。

4） 最近邻（KNN）

K Nearest Neighbor 算法，又称 K 近邻算法。通过计算待分类数据点，与已有数据集中的所有数据点的距离（常见的距离度量有欧式距离，马氏距离等）。对上面所有的距离值进行排序，取距离最小的前 K 个点，根据“少数服从多数”的原则，将这个数据点划分为出现次数最多的那个类别。

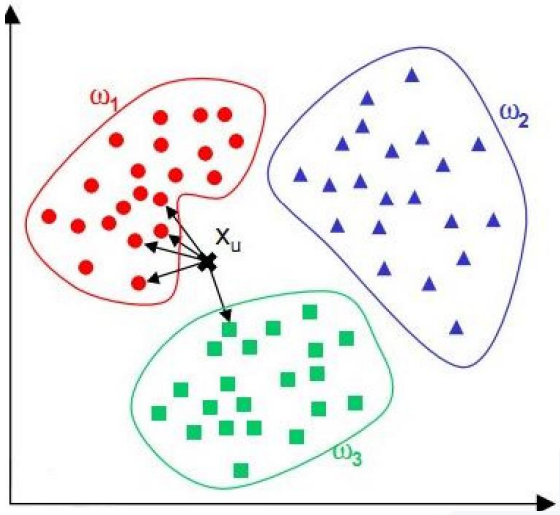


图 1.3.3 KNN 分类图示

如何选择一个最佳的 K 值，这取决于数据。一般情况下，在分类时较大的 K 值能够减小噪声的影响。但会使类别之间的界限变得模糊。一个较好的 K 值可通过各种启发式技术来获取，比如交叉验证。另外噪声和非相关性特征向量的存在会使 K 近邻算法的准确性减小。

KNN 算法的优缺点，优点：思想简单，理论成熟，可以用来做分类也可以做回归；

可用于非线性分类；训练时间复杂度为 O(n)；缺点：计算量大；样本不平衡问题（即有些类别的样本数量很多，而其它样本的数量很少）；需要大量的内存；

3. 分类评价标准

一个模型预测的结果有四种情况如下：

(1) 实际为正例，被预测为正例，预测正确，记为 TP（True Positive）。

(2) 实际为负例，被预测为正例，预测错误，记为 FP（False Positive）。

(3) 实际为正例，被预测为负例，预测错误，记为 FN（False Negative）。

(4) 实际为负例，被预测为负例，预测正确，记为 TN（True Negative）。

在分类评价标准中有如下常用指标：

(1) Precision，精确率：P=TP/（TP+FP）。

(2) Recall，召回率：R=TP/（TP+FN）。

(3) F-Score,即 precision 和 recall 的调和平均值,更接近 precision 和 recall 中较小的那一个值：F=(2*P*R)/(P+R)。

(4) Accuracy,准确性，分类器对整体样本的分类能力，也就是正例分为正例，负例分为负例的概率 A=（TP+TN）/（TP+FP+TN+FN）。

(5) ROC(Receiver Operating Characteristic)，主要用于画 ROC 曲线（横坐标为 FPR，纵坐标为 TPR）。

(6) FPR=FP/(FP+TN),也就是负例被错误预测为正例的数目占总负例的比例。

(7) TPR=TP/（TP+FN），也就是被正确预测的正例占总正例的比例。

1.3.4 实验步骤

使用朴素贝叶斯分类器对新闻主题进行分类，先介绍数据 fetch_20newsgroups，这是一个新闻文字文本数据集，有 20 个分类，样品总数为 18846。

1） 加载新闻数据集，在 sklearn 库中通过如下代码来加载 fetch_20newsgroups。

```
#从 sklearn.datasets 里导入新闻数据抓取器 fetch_20newsgroups
from sklearn.datasets import fetch_20newsgroups
#需从互联网下载数据
news=fetch_20newsgroups(subset='all')
#查验数据规模和细节`
print (len(news.data))
print(news.data[0])
```

数据集正常加载后，查验结果如图 1.3.4 所示：

```
18846
From: Mamatha Devineni Ratnam <mr47+@andrew.cmu.edu>
Subject: Pens fans reactions
Organization: Post Office, Carnegie Mellon, Pittsburgh, PA
Lines: 12
NNTP-Posting-Host: po4.andrew.cmu.edu



I am sure some bashers of Pens fans are pretty confused about the lack
of any kind of posts about the recent Pens massacre of the Devils. Actually,
I am  bit puzzled too and a bit relieved. However, I am going to put an end
to non-PIttsburghers' relief with a bit of praise for the Pens. Man, they
are killing those Devils worse than I thought. Jagr just showed you why
he is much better than his regular season stats. He is also a lot
fo fun to watch in the playoffs. Bowman should let JAgr have a lot of
fun in the next couple of games since the Pens are going to beat the pulp out of
lose the final
regular season game.          PENS RULE!!!
```

图 1.3.4 数据集查验结果

2） 分割数据集，代码如下：

```
#数据分割
from sklearn.model_selection import train_test_split
#随机采样 25%的数据样本作为测试集
X_train,X_test,y_train,y_test=train_test_split(news.data,news.target,test_size=0.25,random_state=33)
```

3） 初始化朴素贝叶斯，使用训练集数据对分类器进行训练，代码如下：

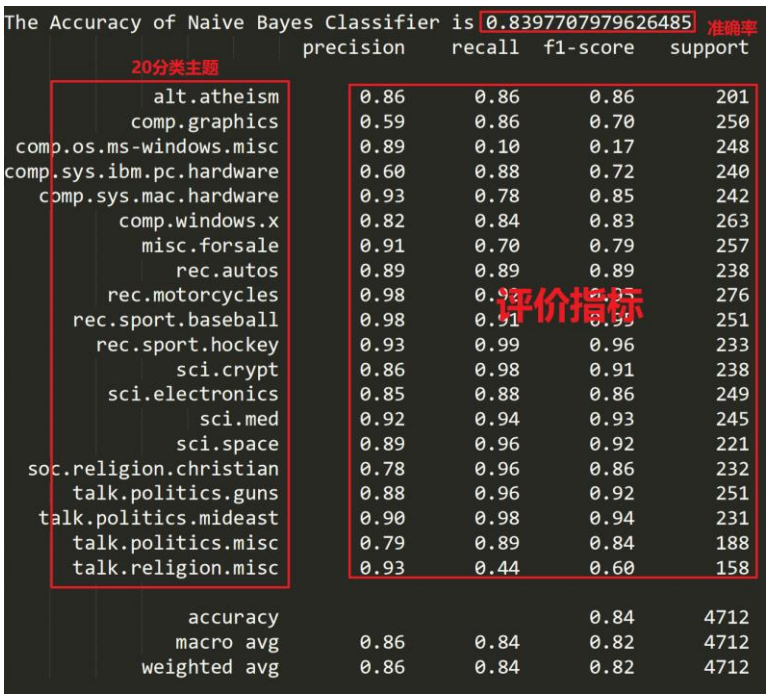
```
#使用朴素贝叶斯分类器进行预测
from sklearn.feature_extraction.text import CountVectorizer
vec=CountVectorizer()
X_train=vec.fit_transform(X_train)
X_test=vec.transform(X_test)
#从 sklearn.naive_bayes 里导入朴素贝叶斯模型
from sklearn.naive_bayes import MultinomialNB
#初始化朴素贝叶斯模型
mnb=MultinomialNB()
#使用  训练数据对模型参数进行估计
mnb.fit(X_train,y_train)
#对测试样本进行预测额，结果保留在储存变量 y_predictio 中
y_predict=mnb.predict(X_test)
```

4） 对朴素贝叶斯分类器在新闻文本数据上的表现进行精度评估，代码如下：

```
#性能评估
```

```
from sklearn.metrics import classification_report
print('The Accuracy of Naive Bayes Classifier is',mnb.score(X_test,y_test))
print(classification_report(y_test,y_predict,target_names=news.target_names))
```

最终使用朴素贝叶斯分类器在该数据上的表现还是比较不错的，训练耗时不到 7 秒，在测试集上模型的分类准确达到了 84%。程序运行结果如图 1.3.5 所示：



```
The Accuracy of Naive Bayes Classifier is 0.8397707979626485
                          precision    recall  f1-score   support
           alt.atheism        0.86      0.86      0.86       201
         comp.graphics        0.59      0.86      0.70       250
comp.os.ms-windows.misc       0.89      0.10      0.17       248
comp.sys.ibm.pc.hardware      0.60      0.88      0.72       240
   comp.sys.mac.hardware      0.93      0.78      0.85       242
          comp.windows.x      0.82      0.84      0.83       263
            misc.forsale      0.91      0.70      0.79       257
               rec.autos      0.89      0.89      0.89       238
         rec.motorcycles      0.98      0.97      0.97       276
      rec.sport.baseball      0.98      0.91      0.99       251
        rec.sport.hockey      0.93      0.99      0.96       233
               sci.crypt      0.86      0.98      0.91       238
         sci.electronics      0.85      0.88      0.86       249
                 sci.med      0.92      0.94      0.93       245
               sci.space      0.89      0.96      0.92       221
  soc.religion.christian      0.78      0.96      0.86       232
      talk.politics.guns      0.88      0.96      0.92       251
   talk.politics.mideast      0.90      0.98      0.94       231
      talk.politics.misc      0.79      0.89      0.84       188
      talk.religion.misc      0.93      0.44      0.60       158

              accuracy                           0.84      4712
             macro avg        0.86      0.84      0.82      4712
          weighted avg        0.86      0.84      0.82      4712
```


图 1.3.5 朴素贝叶斯评估结果

1.3.5 思考题

1. 简述常用的分类算法有哪些？
2. 尝试使用分类算法实现对自定义文本的分类任务。

1.4 决策树

1.4.1 实验目的

1. 学习决策树的基本概念。
2. 学习剪枝方法的实现。
3. 掌握决策树的构造方法和基本流程。
4. 掌握使用 scikit-learn API 函数实现决策树算法。

1.4.2 实验内容

使用 scikit-learn API 中的决策树算法对鸢尾花数据集进行分类预测。

1.4.3 实验原理

1. 决策树简介

决策树（Decision Tree）是一种以树形数据结构来展示决策规则和分类结果的模型，它是将看似无序、杂乱的已知实例，通过某种技术手段将它们转化成可以预测未知实例的树状模型。决策树是一种监督学习的分类算法，要求输入标注好类别的训练样本集，每个训练样本由若干个用于分类的特征来表示。决策树算法的训练目的在于构建决策树，希望能够得到一颗可以将训练样本按其类别进行划分的决策树。假设现在我们想预测的是，女性到底想要嫁什么样的人？我们现在手里拥有一些未婚男性的数据，其中包括了收入、房产、样貌、学历等字段。在构建决策树时，每次都要选择区分度最高的特征，使用其特征值对数据进行划分，每次消耗一个特征，不断迭代，直到所有特征均被使用为止。如果还未使用全部特征，剩下的训练样本就已经具有相同类别了，则决策树的构建可以提前完成。如果使用全部特征后，剩下的训练样本中仍然包含一个以上的类别，则选择剩下的训练样本中占比最大的类别作为这批训练样本的类别。利用决策树的思想，首先我们要考虑的是，上述哪些条件在女性选择男友时最重要的考量指标？若比较在意收入、比较在意物质，则构建的决策树如图 1.4.1 所示：

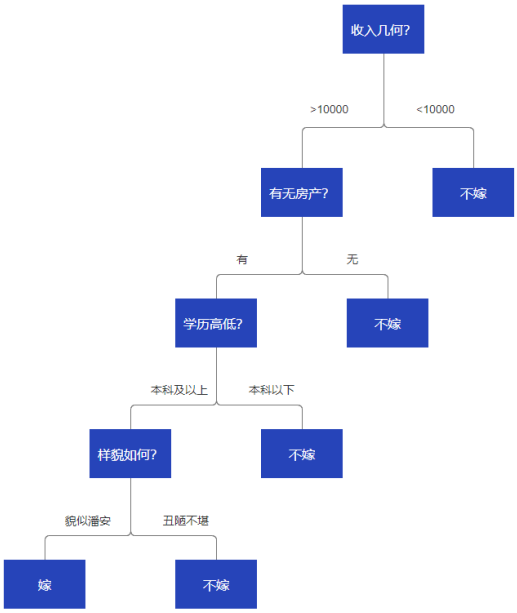


图 1.4.1 决策树

2. 构造及基本流程

使用决策树做预测需要以下过程：

1）收集数据：可以使用任何方法。比如想构建一个相亲系统，我们可以从媒婆那里，或者通过采访相亲对象获取数据。

2）准备数据：收集完的数据，我们要进行整理，将这些所有收集的信息按照一定规则整理出来，并排版，方便我们进行后续处理。

3）分析数据：可以使用任何方法，决策树构造完成之后，我们可以检查决策树图形是否符合预期。

4）训练算法：这个过程也就是构造决策树，同样也可以说是决策树学习，就是构造一个决策树的数据结构。

5）测试算法：使用经验树计算错误率。当错误率达到了可接收范围，这个决策树就可以投放使用了。

6）使用算法：此步骤可以使用适用于任何监督学习算法，而使用决策树可以更好地理解数据的内在含义。

对于有很多特征的数据集，决策树先根据一个特征进行分类，虽然分类结果达不到理想效果，但是通过这次分类，问题规模变小了，同时分类后的子集相比原来的样本集

更加易于分类了。然后针对上一次分类后的样本子集，递归这个过程。在理想的情况下，经过多层的决策分类，将得到完全纯净的子集，也就是每一个子集中的样本都属于同一个分类。

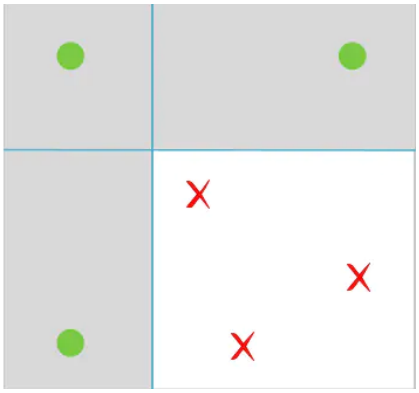


图 1.4.2 样本

图 1.4.2 中，平面坐标中的六个点，无法通过其 x 坐标或者 y 坐标直接就将两类点分开。采用决策树算法思想：先依据 y 坐标将六个点划分为两个子类（如水平线所示），水平线上面的两个点是同一个分类，但是水平线之下的四个点是不纯净的。但是没关系，我们对这四个点进行再次分类，这次我们以 x 左边分类（见图中的竖线），通过两层分类，实现了对样本点的完全分类。决策树的伪代码实现如下：

```
if y > a:
    output dot
else:
    if x < b:
        output cross
    else:
        output dot
```

由这个分类的过程形成一个树形的判决模型，树的每一个非叶子节点都是一个特征分割点，叶子节点是最终的决策分类。如图 1.4.3 所示：

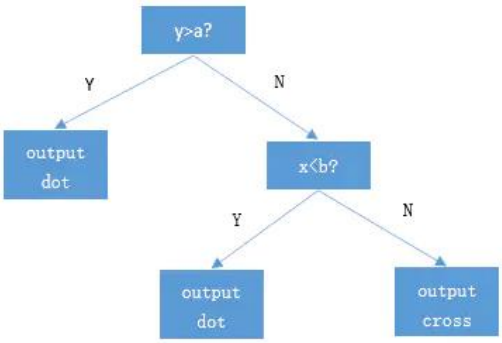


图 1.4.3 决策树模型

将新样本输入决策树进行判决时，就是将样本在决策树上自顶向下，依据决策树的节点规则进行遍历，最终落入的叶子节点就是该样本所属的分类。

策树算法的思想，可以简单归纳为如下两点：

1) 选取特征，分割样本集。
2) 计算增益，如果增益够大，将分割后的样本集作为决策树的子节点，否则停止分割。
3) 递归执行上两步；

3. 剪枝方法

剪枝是指将一颗子树的子节点全部删掉，根节点作为叶子节点，如图 1.4.4 所示：

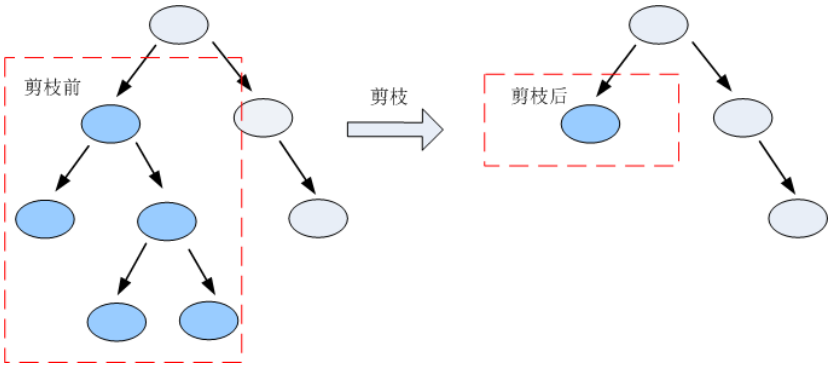


图 1.4.4 决策树剪枝

决策树剪枝的目的是防止构建的决策树出现过拟合。随着决策树的深度的增加，模型的准确度肯定会越来越好。但是对于新的未知数据，模型的表现会很差，泛化能力不够。在数据集没有足够多的情况下，数据集本身存在噪声，同时数据的特征属性不能完全作为分类的标准。以上三点会使决策树出现过拟合现象。

剪枝分为预剪枝和后剪枝两种方法，预剪枝是指在构建完全正确分类训练集 的决策树之前，停止树的构建。常见有 3 种方法来决定何时停止树的构建：

1) 预设树的高度。当决策树的高度达到预设值之后，停止继续构建。
2) 设定阈值。当叶子结点里的实例数量小于阈值时，停止。
3) 设定阈值。计算每一次增加深度后模型的增益，增益小于阈值，则停止。

预剪枝优点是速度快，计算量少。缺点是视线短浅。比如一颗完整的决策树有 5 层，A->B->C->D->E。从 B->C 的过程中模型几乎没有什么提升，但是从 C->D 的过程中模型的准确度提升显著。这种情况使用预剪枝，会使模型提前终止。

后剪枝的整体思路是先构建完整的决策树，然后再对决策树进行剪枝操作。也有 3 种常用的剪枝方法：

1） 错误率降低剪枝：

使用一个测试集，对于非叶子节点的子树，尝试把它替换成叶子节点。用子树中样本数量最多的类来表示这个节点的结果。比较替换前后，两个决策树在测试集上的表现。从下至上，遍历所有的可能的子树，直到在测试集上没有提升时，停止。

2） 悲观剪枝：

对于决策树中的子树，尝试把它直接替换成一个叶子节点（具体用哪个节点来替代不太确定，有些资料表示直接用子树的根来替换）。比较 被替换子树的错误数-标准差（由二项分布计算） 和 新叶子节点错误数，如果前者大，那么执行剪枝操作。反之保留。

3） 代价复杂度剪枝：

代价复杂度选择节点表面误差率增益值最小的非叶子节点，删除该非叶子节点的左右子节点，若有多个非叶子节点的表面误差率增益值相同小，则选择非叶子节点中子节点数最多的非叶子节点进行剪枝。

1.4.4 实验步骤

开始做一个使用决策树对鸢尾花数据集进行分类预测实验，数据鸢尾花集是常用的分类实验数据集，由 Fisher,1936 收集整理。Iris 也称鸢尾花卉数据集，是一类多重变量分析的数据集。数据集包含 150 个数据集，分为 3 类，每类 50 个数据，每个数据包含 4 个属性。iris 以鸢尾花的特征作为数据来源，常用在分类操作中。该数据集由 3 种不同类型的鸢尾花的 50 个样本数据构成。其中的一个种类与另外两个种类是线性可分离的，后两个种类是非线性可分离的。

四个属性：

1） Sepal.Length（花萼长度），单位是 cm;
2） Sepal.Width（花萼宽度），单位是 cm;
3） Petal.Length（花瓣长度），单位是 cm;
4） Petal.Width（花瓣宽度），单位是 cm。

三个种类：

1) Iris Setosa（山鸢尾）；

2) Iris Versicolour（杂色鸢尾）；

3) Iris Virginica（维吉尼亚鸢尾）。

1) 建立工程导入必要的库，导入鸢尾花数据集。

```
import sklearn.datasets as datasets
from sklearn.model_selection import train_test_split
# 导入鸢尾花数据集
iris = datasets.load_iris()

iris_feature = iris.data #特征数据
iris_target = iris.target #分类数据
```

2) 分割数据集。

```
"""
        feature_train, feature_test, target_train, target_test 分别代表训练集特征、测试集特征、训练集目标值、验证集特征。
        test_size 参数代表划分到测试集数据占全部数据的百分比，你也可以用 train_size 来指定训练集所占全部数据的百分比。
        一般情况下，我们会将整个训练集划分为 70% 训练集和 30% 测试集。最后的 random_state 参数表示乱序程度。

"""
feature_train, feature_test, target_train, target_test = train_test_split(iris_feature, iris_target, test_size=0.33, random_state=42)
```

3) 训练决策树模型。

```
from sklearn.tree import DecisionTreeClassifier,export_graphviz
"""
DecisionTreeClassifier() 模型方法中也包含非常多的参数值。例如：

criterion = gini/entropy 可以用来选择用基尼指数或者熵来做损失函数。
splitter = best/random 用来确定每个节点的分裂策略。支持“最佳”或者“随机”。
max_depth = int 用来控制决策树的最大深度，防止模型出现过拟合。
min_samples_leaf = int 用来设置叶节点上的最少样本数量，用于对树进行修剪。
"""
dt_model = DecisionTreeClassifier() # 所以参数均置为默认状态
dt_model.fit(feature_train,target_train) # 使用训练集训练模型
predict_results = dt_model.predict(feature_test) # 使用模型对测试集进行预测
```

4) 模型评价。

```
from sklearn.metrics import accuracy_score
print(accuracy_score(predict_results, target_test))
```

最终决策树模型在测试集上的精度表现为 98%。

5） 将决策树模型进行可视化。

```
import pydotplus
import os
os.environ["PATH"] += os.pathsep + 'C:/Program Files (x86)/Graphviz2.38/bin/' #注意修路径
dot_data = export_graphviz(dt_model,out_file=None,feature_names=iris.feature_names,class_names=iris.target_names,filled=True,rounded=True,special_characters=True)
graph = pydotplus.graph_from_dot_data(dot_data)
graph.write_png('iris.png')
```

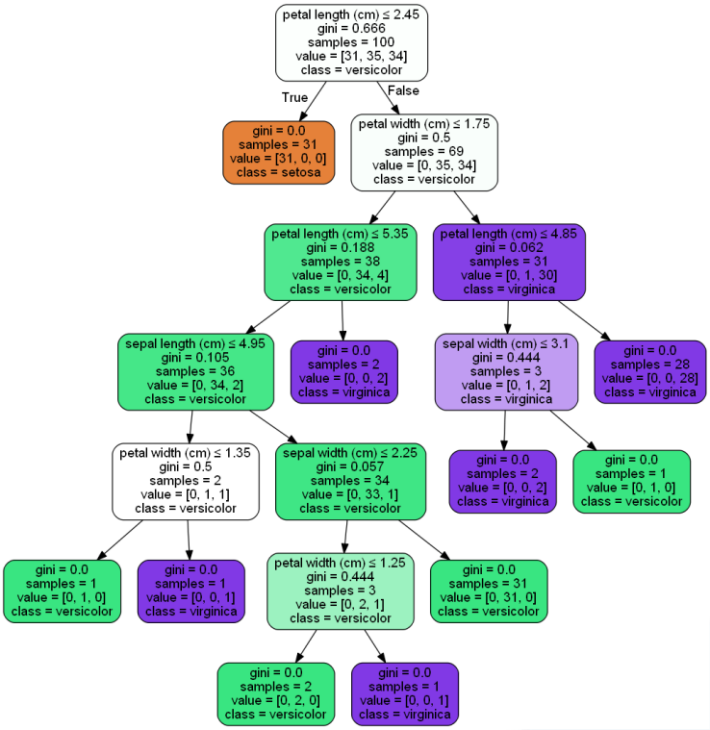


图 1.4.5 决策树模型可视化

1.4.5 思考题

1. 简要阐述决策树算法的实现原理及构造流程。
2. 思考使用决策树算法还能解决哪些问题？

1.5 支持向量机 SVM

1.5.1 实验目的

1. 学习支持向量机 SVM 的基本概念。
2. 了解核函数的基本概念。
3. 掌握使用 scikit-learn API 函数实现 SVM 算法。

1.5.2 实验内容

使用 scikit-learn API 中的 SVM 算法解决非线性分类问题。

1.5.3 实验原理

1. 支持向量机简介

支持向量机(Support Vector Machines,SVM)：支持向量(Support Vector)就是离分隔超平面最近的那些点，机(Machine)就是表示一种算法，而不是表示机器。SVM 是一种监督学习算法，主要用于分类问题，主要的应用场景有字符识别、面部识别、行人检测、文本分类等领域。通常 SVM 用于二元分类问题，对于多元分类通常将其分解为多个二元分类问题，再进行分类。

如图 1.5.1 是典型的二分类问题，明显发现 D 会比 B、C 分割效果要好。SVM 的目标就是找到这样的一个超平面（对于图 1.5.1 来说，就是找到一条直线），使得不同类别的数据能够落在超平面的两侧。

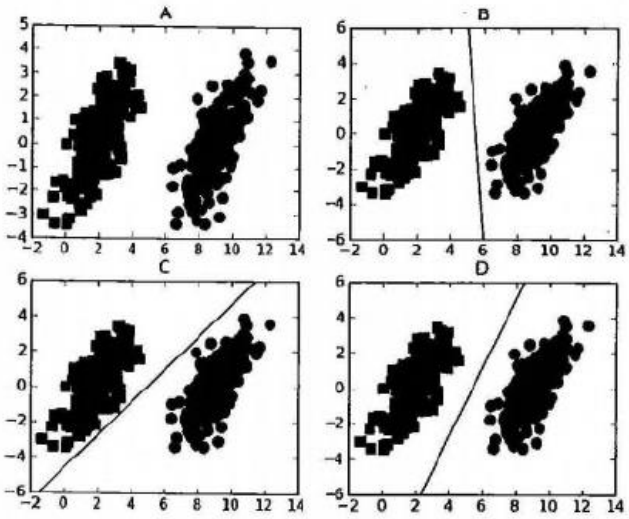


图 1.5.1 A 给出了一个线性可分数据集，B、C、D 各自给出了一条可以将两类数据分开的直线

2. 间隔

图 1.5.1 D 框所示正负样本分得越开越好，代表正负样本之间的几何间隔越大越好。这是因为距离超平面越近的样本，分类的置信度越低（实际上几何间隔代表了分类器的误差上界）。SVM 的优化目标，正是最大化最接近超平面的点到超平面的距离。

我们把这个二维的分类超平面设为 f(x)=wTx+b=0。把样本 x 代入 f(x)中，如果得到的结果小于 0，我们对该样本标一个-1 的类别标签 yi，大于 0 则标一个+1 的 yi。(约定为+1 或-1 只是为了下面的推导便利而已)。

首先定义函数间隔（function margin）：

$\gamma\hat{}=|wTx+b|=yi(wTx+b)$

注意前面乘上 yi 可以保证这个 margin 的非负性（因为 f(x)<0 对应 yi=-1 的那些样本）。

如下图 1.5.2 所示，对任意不在分类超平面上的点 xi，我们可以依赖它到分类超平面的垂直投影 x0，计算出它到分类超平面上的几何间隔（geometrical margin）：

$\gamma\tilde{}=|wTxi+b| / ||w||=yi(wTx+b)| / |w||=\gamma\hat{} / ||w||$

（||w||是向量 w 的范数，是对 w 长度的一种度量）。于是我们得到了函数间隔和几何间隔的数值关系。如图 4-17 所示：

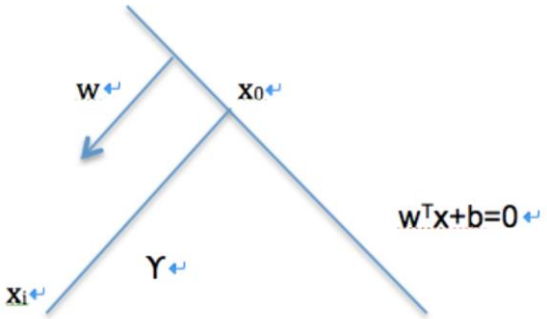


图 1.5.2 几何间隔

$\gamma\hat{}$实际上不影响 SVM 优化问题的求解，为了简化问题，我们设 $\gamma\hat{}=1$，从而把优化问题转化为：$\max\gamma\tilde{}=1/||w||, s.t. yi(wTx+b)>=1, i=1,\cdots,n$。

这个问题可以转化为一个等价的二次规划问题，也就是说它必然能得到一个全局的最优解。通过求解这个问题，我们可以得到了一个最大化几何间隔的分类超平面（如图 1.5.3 红线所示），另外两条线到红线的距离都等于 1/||w||，而橘黄色的样本就是支持

向量 Support Vector。

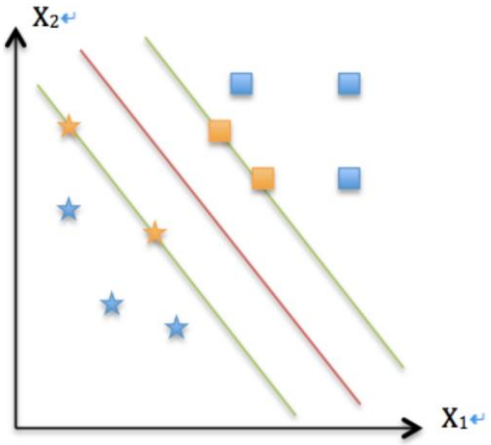


图 1.5.3 SVM 分类示意

通过最大化几何间隔，使得该分类器对样本分类时有了最大的置信度，准确的说，是对置信度最小的样本有了最大的置信度，这正是 SVM 的核心思想。

3. 核函数与方法

对于简单的线性可分数据可以使用软间隔 SVM 或硬间隔 SVM 来划分，对于复杂的线性可分和线性不可分数据使用核函数，通过将特征向量映射到更高维的空间中，使得原本线性不可分数据在高维空间中变得线性可分。如图 1.5.4 所示，A 是一堆线性不可分数据，B 是通过 SVM 核函数将特征向量拓展到高维空间中使得数据可分。绿色的平面就是 SVM 要找的分割平面。C 是 B 的鸟瞰图。

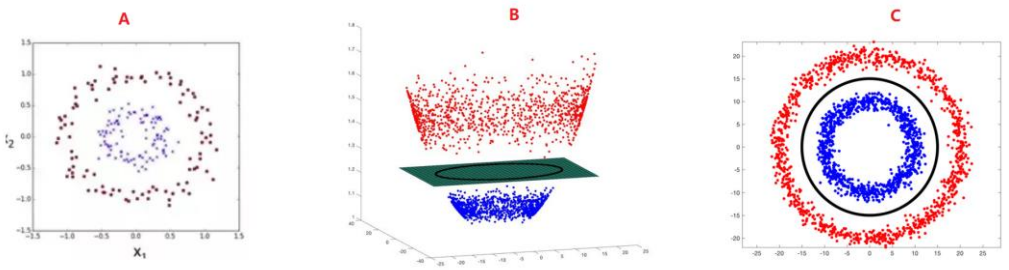


图 1.5.4 高维拓展

核函数解决非线性可分数据的核心思想是，将原始数据样本通过核函数映射到更高的维度空间中，直到样本在高维度空间中是线性可分的。然后再使用常见的线性分类器分割样本数据。由于拓展到了高维空间，引入核函数就是为了降低向量内积的计算量。

常见的核函数如图 1.5.5 所示：

名称	表达式	参数
线性核	$k(x_i, x_j) = x_i^T x_j$	
多项式核	$k(x_i, x_j) = (x_i^T x_j)^d$	$d \geq 1$为多项式的次数
高斯核	$k(x_i, x_j) = exp(-\frac{\|\|x_i - x_j\|\|^2}{2\sigma^2})$	$\sigma > 0$为高斯核的带宽（width）
拉普拉斯核	$k(x_i, x_j) = exp(-\frac{\|\|x_i - x_j\|\|}{\sigma})$	$\sigma > 0$
Sigmoid 核	$k(x_i, x_j) = tanh(\beta x_i^T x_j + \theta)$	tanh为双曲正切函数，$\beta > 0, \theta < 0$

图 1.5.5 常见 SVM 核函数

对于线性可分数据，优先使用线性核，对于线性可分数据可以使用线性核，但不推荐使用，对于线性不可非数据，只能使用非线性核，不能使用线性核。线性核（linear kernel）和高斯核（RBF kernel）在线性可分和不可分数据划分可视化对比如图 1.5.6 所示：

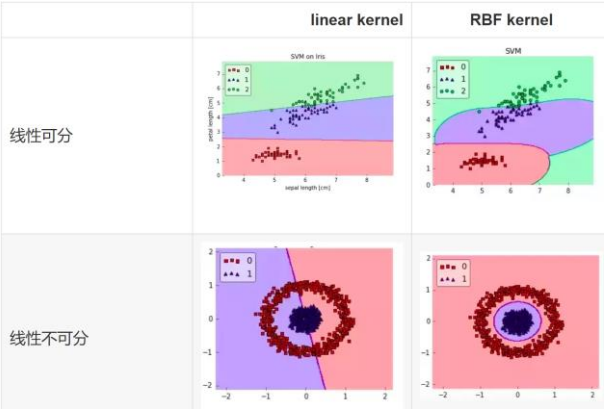


图 1.5.6 不同核函数划分效果

1.5.4 实验步骤

使用 SVM 解决非线性分类问题步骤如下：

1） 建立工程并导入 sklearn 库。

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
```

2） 随机生成非线性数据。

```
xx, yy = np.meshgrid(np.linspace(-3, 3, 500),
```

```
                                    np.linspace(-3, 3, 500))
np.random.seed(0)
X = np.random.randn(300, 2)
Y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
```

3） 创建 SVM 预测模型。

```
clf = svm.NuSVC(gamma='auto')
clf.fit(X, Y)
```

4） 为网格上的每个数据点绘制决策函数，运行结果如图 1.5.7 所示。

```
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.imshow(Z, interpolation='nearest',
                extent=(xx.min(), xx.max(), yy.min(), yy.max()), aspect='auto',
                origin='lower', cmap=plt.cm.PuOr_r)
contours = plt.contour(xx, yy, Z, levels=[0], linewidths=2,
                                linestyles='dashed')
plt.scatter(X[:, 0], X[:, 1], s=30, c=Y, cmap=plt.cm.Paired,
                 edgecolors='k')
plt.xticks(())
plt.yticks(())
plt.axis([-3, 3, -3, 3])
plt.show()
```

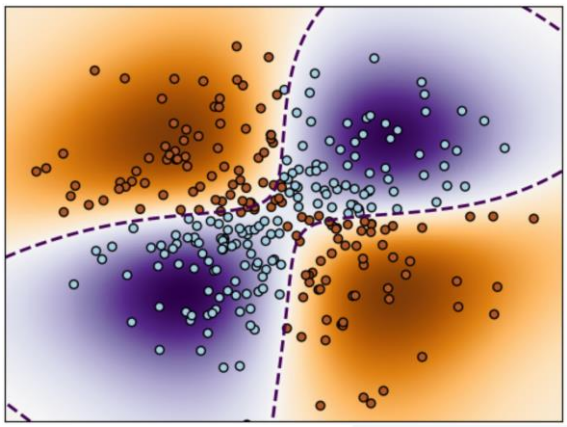


图 1.5.7 非线性分类

1.5.5 思考题

1. 简要阐述决策树算法的实现原理及构造流程。
2. 思考使用决策树算法还能解决哪些问题？